
Expected Results

External specifications and baselines

Specifications, requirements etc. define what the software is required to do

In agile dev environments, requirements may just be user stories or whiteboard scribbles.

Without requirements, developers cannot build, testers cannot test

Programmers need them to write the code

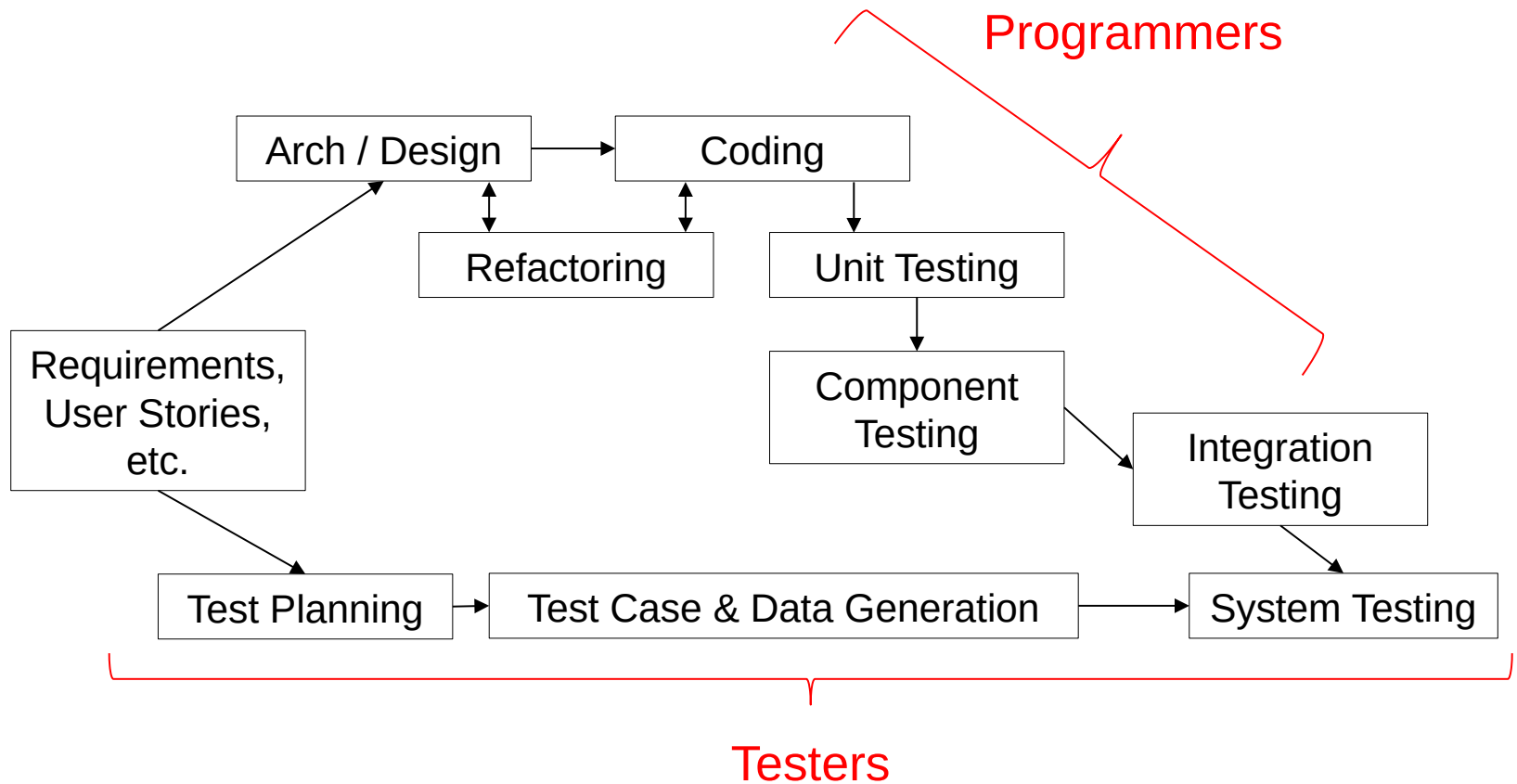
Testers need them to:

- identify the things that need testing

- compare test results with requirements

Requirements, specs. Etc. are baselines.

Testing Sequencing



Baseline as an oracle¹ for required behaviour

When we test we get an actual result

We compare results with requirements to determine whether a test has passed

A **baseline** document describes how we require the system to behave

User requirement, design, spec. etc.

Sometimes the 'old system' tells us what to expect.

¹ A source to determine expected results to compare with the actual result of the software under test. An oracle may be the existing system (for a benchmark), a user manual, or an individual's specialized knowledge, but should not be the code.

Expected results

If we don't define expected result before we execute the test...

- a plausible, but erroneous, result may be interpreted as the correct result

- there may be a subconscious desire to see the software pass the test

Expected results must be defined before test execution, derived from a baseline.

Exit, Completion, Closure or Acceptance criteria

Exit Criteria

Trigger to say: "we've done enough"

Objective, non-technical for managers

Measurable, achievable target. Some typical types of criterion which are used regularly are listed below:

- 80% coverage achieved

- all tests executed without failure

- all outstanding incidents waived

- all critical business scenarios covered.

Exit criteria 2

Coverage items are usually defined in terms of requirements, conditions, business transactions code statements, branches.